

一种基于加权区域采样的直线反走样生成算法

杭后俊¹, 付 勇²

(1. 安徽师范大学 数学计算机科学学院, 安徽 芜湖 241000;
2. 合肥工业大学 计算机科学技术系, 安徽 合肥 230009)

摘 要: 直线反走样生成算法是光栅化图形算法中的重要内容。主要讨论了一种离散化的加权区域采样方法。将一个像素均匀分割成若干个子像素, 根据每个子像素在像素中的位置赋予它们一定的权值。讨论了如何计算一个像素对线段的贡献(也即是该像素的灰度)。对Bresenham画线算法进行了改进, 给出了一种反走样算法。从实验的结果看, 该算法具有比较明显的反走样效果; 和传统的反走样算法相比, 文中算法效率更优。

关键词: 光栅化; Bresenham画线算法; 反走样; 加权区域采样算法

中图分类号: TP 301.6 **文献标识码:** A **文章编号:** 1673-629X(2009)06-0138-04

One Antialiasing Algorithm Based on Weighting Region Sampling

HANG Houjun¹, FU Yong²

(1. School of Mathematics and Computer Science, Anhui Normal University, Wuhu 241000, China;
2. Dept. of Computer Science and Technology, Hefei University of Technology, Hefei 230009, China)

Abstract: Antialiasing algorithm is very important content in graph algorithm. Discusses mainly a weighting area sampling. First, divide a pixel into some small pixel, then distribute these small pixels different weighting value according to their position in pixel. Second, discuss how to calculate a pixel's contribution to line and give corresponding algorithm. Finally improve Bresenham algorithm and give an antialiasing algorithm. From the result, the algorithm has obvious antialiasing effect. Compared with the classical method, the presented method is more efficient.

Key words: rasterization; Bresenham algorithm; antialiasing; weighting region sampling

0 引 言

Bresenham画线算法是非常流行的直线光栅化生成算法。该算法具有效率高、易于硬件实现等优点。但光栅化后的直线(段)和理想的直线不同, 它是有单像素宽度的。这就使得光栅化后的直线(水平、垂直直线除外)有锯齿, 即所谓的“走样”^[1~4]。

多年来对如何消除这种走样现象提出了许多相关算法^[5~9], 加权区域采样算法就是其中比较有代表性的算法之一。其基本思想是, 当直线经过某像素时, 该像素的亮度值是在直线与像素所交区域上对滤波函数进行积分所得到的积分值。由于求积分的运算量很大, 所以可以通过选取有代表性的直线组成采样数组以减少运算量^[9], 但各像素权值取值的准确性很难保证。文中给出了一种基于加权区域采样算法的离散化

方法——加权平均的子像素划分方法, 并基于该技术对Bresenham画线算法进行了改进, 与传统的离散化反走样算法相比, 实现效果完全相同但效率更优。

1 像素的均匀分割

将一个像素在水平与垂直方向上均匀分割为 n 个子像素, 则每个子像素的面积为 $1/n$ 。再计算每个子像素对原像素灰度的贡献值, 并保存在一张表(称为加权表)中。一般来说, 离像素中心越近的子像素其贡献值就大(这和滤波函数的特点相吻合)。在实际应用中, 经常将一个像素划分为 9 个子像素, 其加权表可取为

$$\begin{bmatrix} w_1 & w_2 & w_3 \\ w_4 & w_5 & w_6 \\ w_7 & w_8 & w_9 \end{bmatrix} = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

2 子像素中心与矩形区域位置关系的判断

因为一个像素宽度的直线段实际上可以看作是一

收稿日期: 2008-09-19; 修回日期: 2008-12-29

基金项目: 安徽省自然科学基金(2006kj076B)

作者简介: 杭后俊(1965-), 男, 安徽无为, 副教授, 研究方向为计算机图形学、CAD等。

个宽度为1的矩形区域。当直线与某一个像素相交时,关键问题是如何求得该像素的哪些子像素的中心落在矩形区域中。

设理想直线段 l 的两个端点为 $P(x_1,y_1)$ 和 $P(x_2,y_2)$ 。则其一般方程为

$$ax+by+c=0$$

其中 $a=y_0-y_1, b=x_1-x_0, c=x_0y_1-x_1y_0$ 。

由 l 决定的平行直线束的方程为 $ax+by+u=0(u\in R)$ 。

根据两平行线间距离公式知:

$$\frac{|u-c|}{\sqrt{a^2+b^2}}=\frac{1}{2}$$

解出: $u_1=\frac{1}{2}\sqrt{a^2+b^2}+c$ 或

$$u_2=-\frac{1}{2}\sqrt{a^2+b^2}+c$$

不妨设 l_1 的方程为 $ax+by+u_1=0, l_2$ 的方程为 $ax+by+u_2=0$

令 $F(x,y)\equiv ax+by+u_1, F(x,y)=ax+by+u_2$

对 $\forall M(x_0,y_0)$,由解析几何知识可知:

①若 $F(x_0,y_0)$ 与 $F(x_0,y_0)$ 异号,则 M 在区域内部;

②若 $F(x_0,y_0)$ 与 $F(x_0,y_0)$ 同号,则 M 在区域外部;

③若 $F(x_0,y_0)$ 与 $F(x_0,y_0)$ 中有一个为0,则 M 在区域边界上。在算法实现时,也认为 M 在区域内部。

因为 $F(x_0,y_0)\cdot F(x_0,y_0)=(ax_0+by_0+\frac{1}{2}\cdot\sqrt{a^2+b^2}+c)(ax_0+by_0-\frac{1}{2}\cdot\sqrt{a^2+b^2}+c)=(ax_0+by_0+c)^2-\frac{1}{4}(a^2+b^2)$

令 $d(x,y)=4(ax+by+c)^2-(a^2+b^2)$,则上述判断方法可以等价地写成:

对 $\forall M(x_0,y_0)$,①若 $d(x_0,y_0)\leq 0$,则 M 在区域内部;②若 $d(x_0,y_0)>0$,则 M 在区域外部。

3 主要算法思想

假设直线的斜率在0到1之间,对于斜率不在0到1之间的直线(段),可对其进行一种对称变换,找到它的斜率在0到1之间的对称直线(段)进行处理。如图1所示,在用Bresenham画线算法生成直线(段)过程中,假设当前最佳逼近线段的像素是 P 。因为一个像素宽度的直线段实际上可以看作是一个宽度为1的矩形区域。因此不难理解,像素 P 的正上方像素 T 和正下

方像素 S 都有可能与该矩形区域相交。

因此,在算法中应该分别计算三个像素落在矩形区域中的子像素的位置及数量。利用加权表计算出该像素的灰度值。

假设当前处理的像素坐标为 (x,y) ,不妨将该像素“均匀”划分为 3×3 个子像素(如图2所示),则各个子像素的中心坐标如下:

子像素1中心坐标: $(x-1/3,y+1/3)$

子像素2中心坐标: $(x,y+1/3)$

子像素3中心坐标: $(x+1/3,y+1/3)$

子像素4中心坐标: $(x-1/3,y)$

子像素5中心坐标: (x,y)

子像素6中心坐标: $(x+1/3,y)$

子像素7中心坐标: $(x-1/3,y-1/3)$

子像素8中心坐标: $(x,y-1/3)$

子像素9中心坐标: $(x+1/3,y-1/3)$

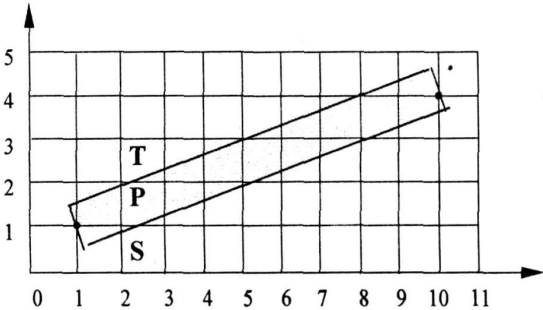


图1 当前像素的正上(下)方像素可能与矩形区域相交

1	2	3
4	5	6
7	8	9

图2 子像素及编号

在具体的算法中,不必判断每个子像素与矩形区域的位置关系,为了提高算法得效率,对像素 P 、像素 T 和像素 S 分别进行处理。

1) 对像素 P ,如果其子像素1(的中心),子像素9(的中心)均落在矩形区域中,则所有子像素均落在矩形区域中。

2) 对像素 T ,如果子像素5位于矩形区域之外,如图3所示。因为直线的斜率在0到1之间,容易得出子像素1,2,3,4均在矩形区域之外。只需判断子像素6,7,8,9与矩形区域的位置关系,将其中落在矩形区域中的子像素的权值进行累加,再以该值乘以当前像素的灰度值。

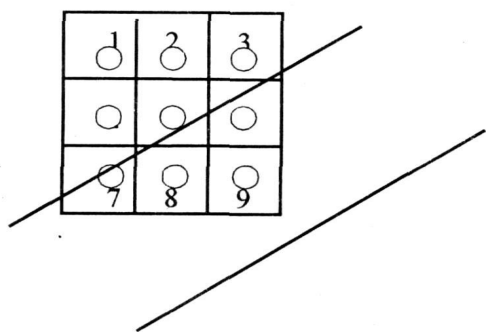


图 3 子像素 5 位于矩形区域之外 I

3) 对像素 S , 如果子像素 5 位于矩形区域之外, 如图 4 所示。因为直线的斜率在 0 到 1 之间, 容易得出子像素 6, 7, 8, 9 均在矩形区域之外。只需判断子像素 1, 2, 3, 4 与矩形区域的位置关系, 将其中落在矩形区域中的子像素的权值进行累加, 再以该值乘以当前像素的灰度值。

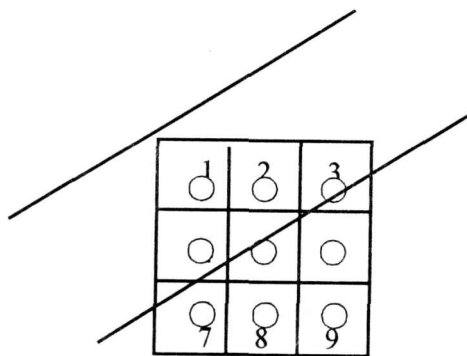


图 4 子像素 5 位于矩形区域之外 II

通过上面的讨论, 下面给出文中算法的详细代码。为此, 先定义一个全局数组 s 和全局变量 a, b, c 。

$\text{int } s[3][3] = \{1, 2, 1, 2, 4, 2, 1, 2, 1\};$ // 存储子像素的权值

$\text{int } a, b, c;$ // 保存理想直线方程中的一次项系数和常数项

算法 1: 计算当前像素灰度值。

$\text{float pixel_value}(x, y)$

$\text{int } x, y;$

$\{ \text{int } i, j;$

$\text{static int } k = 0;$

$\text{float } q = 0.0, d, d1, d2, t1, t2;$

$\text{if}(k == 1)$ // 如果当前处理的像素是 P

$\{ t1 = a * (x - 1.0/3) + b * (y + 1.0/3) + c;$

$d1 = 4 * t1 * t1 - a * a - b * b;$

$t2 = a * (x + 1.0/3) + b * (y - 1.0/3) + c;$

$d2 = 4 * t2 * t2 - a * a - b * b;$

$\text{if}(d1 \leq 0 \& \& d2 \leq 0)$ // 如果子像素 1 和子像

素 9 均落在矩形区域内

$q = 1;$

else

$q = \text{color_value}(x, y);$

$\}$

$\text{if}(!k)$ // 如果当前处理的像素是 S

$\{ t1 = a * x + b * y + c;$

$d = 4 * t1 * t1 - a * a - b * b;$

$\text{if}(d > 0)$ // 如果像素 5 在矩形区域之外

$\text{for}(i = 0; i < 3; i++)$ // 判断子像素 1, 2, 3 与

矩形区域的位置关系

$\{$

$t1 = a * (x + 1.0/3 * (i - 1)) + b * (y + 1.0/3) +$

$c;$

$d = 4 * t1 * t1 - a * a - b * b;$

$\text{if}(d \leq 0)$ // 如果当前子像素位于矩形区域内

$q += (\text{float}) s[i][2] / 16;$ // 将该子像素的权值

累加至 q 中

$\}$

$t1 = a * (x - 1.0/3) + b * y + c;$

$d = 4 * t1 * t1 - a * a - b * b;$

$\text{if}(d \leq 0)$ // 如果子像素 4 位于矩形区域内

$q += (\text{float}) s[0][2] / 16;$ // 将该子像素的权值

累加至 q 中

$\}$

else // 如果像素 5 在矩形区域之内

$q = \text{color_value}(x, y);$

$\}$

$\text{if}(k == 2)$ // 如果当前处理的像素是 T

$\{ t1 = a * x + b * y + c;$

$d = 4 * t1 * t1 - a * a - b * b;$

$\text{if}(d > 0)$ // 如果像素 5 在矩形区域之外

$\text{for}(i = 0; i < 3; i++)$ // 判断子像素 7, 8, 9 与

矩形区域的位置关系

$\{ t1 = a * (x + 1.0/3 * (i - 1)) + b * (y - 1.0/3) +$

$c;$

$d = 4 * t1 * t1 - a * a - b * b;$

$\text{if}(d \leq 0)$ // 如果当前子像素位于矩形区域内

$q += (\text{float}) s[i][0] / 16;$ // 将该子像素的权值

累加至 q 中

$\}$

$t1 = a * (x + 1.0/3) + b * y + c;$

$d = 4 * t1 * t1 - a * a - b * b;$

$\text{if}(d \leq 0)$ // 如果子像素 6 位于矩形区域内

$q += (\text{float}) s[2][2] / 16;$ // 将该子像素的权值

累加至 q 中

$\}$

else // 如果像素 5 在矩形区域之内

```
q = color -value(x ,y) ;
}
k ++;
if(k ==3)k =0;
return(q) ;
}

float color -value(x ,y)
int x ,y ;
{int i ,j ;
float q =0.0,t ,d ;
for(j =0;j <3;j ++) //判断每个子像素与矩形区域的位置关系
for(i =0;i <3;i ++)
{t =a *(x +1.0/3 *(i -1)) +b *(y +1.0/3 *(j -1)) +c ;
d =4 *t *t -a *a -b *b ;
if(d <=0) //如果当前子像素位于矩形区域内
q +=(float)s[i][j] /16;//将该子像素的权值累加至q 中
}
return(q) ;
}
```

算法 2: 基于加权区域采样的(斜率在 0 到 1 之间) 直线段反走样算法。

```
void Antilise -line(int x 0,int y 0,int x 1,int y 1,int color)
or)
{int d 1,d 2,d ,x ,y ;
a =y 0-y 1,b =x 1-x 0,c =x 0 *y 1-x 1 *y 0,d
=2 *a +b ;
d 1=2 *a ,d 2=2 * (a +b) ;
x =x 0,y =y 0;
for(i =0;i <3;i ++) //计算像素S ,P ,T 的灰度值
{color =color *pixel -value(x ,y +i -1) ;
drawpixel(x ,y +i -1,color) ;
}
while (x <x 1)
{if (d <0)
{x ++,y ++,d +=d 2; }
else {x ++,d +=d 1;}
for(i =0;i <3;i ++) //计算像素S ,P ,T 的灰度值
{color =color *pixel -value(x ,y +i -1) ;
drawpixel(x ,y +i -1,color) ;
}
}
```

4 结束语

给出了算法的 VC 实现。从实现结果可以看出, 采用文中的算法后线段的锯齿明显消失, 取得明显的反走样效果。首先, 相比较使用滤波函数进行积分而言, 文中给出的算法运行效率明显要高。其次, 与传统的离散化反走样算法相比, 实现效果完全相同但效率更优。如图 5 所示, 左边是反走样前的直线段, 中间是传统的反走样效果, 右边是利用文中算法的效果。

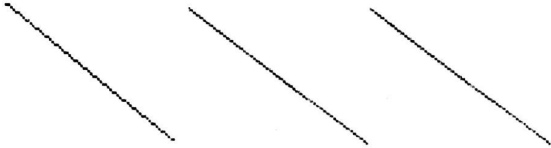


图 5 文中算法和传统算法的比较

表 1 给出了文中算法与传统的离散化反走样算法的比较。从表中可以看出, 文中算法从判断与矩形区域位置关系的子像素数量方面, 执行效率明显要好于传统的离散化反走样算法。

表 1 文中算法与文献 [1] 中算法的效率比较

线段 起点 (10,10))	需要判断的子像素数(个)	
	文献 [1] 中算法	文中算法
终点 (100,50)	2457	1001
终点 (100,70)	2457	1541
终点 (100,90)	2457	1921

参考文献:

[1] 孙家广. 计算机图形学[M] . 北京: 清华大学出版社, 1998.
[2] Donald M. Baker P. Computer Graphics with OpenGL[M] . 3rd ed. [s. l.] : Publishing House of Electronics Industry , 2005: 175—185.
[3] Xiang Zhigang . Computer Graphics with OpenGL[M] . Beijing :Tsinghua University Press , 2006: 63—67.
[4] 潘云鹤, 董金祥, 陈德人. 计算机图形学——原理、方法及应用[M] . 北京: 高等教育出版社, 2001.
[5] 谢莹, 许荣斌, 赵宏坤. 基于嵌入式图形系统的改进 Bresenham 反走样算法[J] . 计算机技术与发展, 2006, 16(11) : 106—108.
[6] Wu X. An Efficient Antialiasing Technique[J] . Computer Graphics , 1991, 25(4) : 143—152.
[7] Kirk D, Arvo J. Unbiased Sampling Techniques for Image Synthesis[J] . Computer Graphics , 1991, 25(4) : 153—156.
[8] Kori en J U, Badler N I. Temporal Antialiasing in Computer - Generated Animation[J] . Computer Graphics , 1983, 17(3) : 377—388.
[9] 刘涛, 高青伟, 赵国荣. 一种新的基于圆锥滤波的直线反走样生成技术[J] . 工程图学报, 2006, 27(3) : 67—70.